# FinSight Cross-Sell Platform

## AI-Powered Intelligent Banking Recommendation System

Team: **Meow Meow**
InnovGenius 2026 · TCS Hackathon · Problem Statement PS-02

| | |
|---|---|
| **Event** | InnovGenius 2026 — TCS Hackathon |
| **Problem Statement** | PS-02: AI-Driven Cross-Sell Recommendation |
| **Team Name** | Meow Meow |
| **Submission** | FinSight Cross-Sell Platform v1.0 |
| **Date** | February 2026 |

# 1. Executive Summary

FinSight is a production-ready, AI-powered cross-sell recommendation platform built for modern retail banking. It replaces generic, spray-and-pray marketing with hyper-personalised, data-driven product offers — delivered at exactly the right moment in a customer's financial journey.

The system ingests real-time transaction data, detects significant life events (home purchase, marriage, education, medical emergency, travel, vehicle purchase), evaluates credit eligibility, selects the most suitable financial product, and generates a compliant, personalised offer message using Google Gemini AI — all within a single API call.

| Metric | Value | Metric | Value |
|---|---|---|---|
| AI Model | Gemini 1.5 Flash | Backend | FastAPI (Python) |
| Database | Supabase (PostgreSQL) | Frontend | React + Vite |
| Life Events Detected | 6 categories | Compliance Checks | 6 automated |
| Credit Score Range | 600 – 850 | Max Loan Multiplier | 5x annual income |

*Table 1: Platform at a Glance*

# 2. Problem Statement

Indian retail banks manage millions of customers across diverse financial products — credit cards, personal loans, home loans, education loans, insurance, and investment products. Despite this breadth, cross-sell conversion rates remain stubbornly low (typically 2–5%) because:

- Offers are generic and not tied to the customer's current financial situation or life stage.
- Marketing campaigns are batch-driven, not real-time — missing the critical moment of intent.
- Risk assessment is manual or rule-based, leading to either over-rejection or under-screening.
- Compliance validation is inconsistently applied, creating regulatory exposure.
- There is no feedback loop: offer performance data does not inform future recommendations.

**PS-02 Challenge:** Build an AI-driven system that analyses customer transaction data, detects life events, assesses credit eligibility, and generates personalised, compliant product recommendations — delivered through the customer's preferred communication channel.

# 3. Solution Overview

FinSight solves the cross-sell problem through a six-stage AI pipeline that runs end-to-end in a single API request, with results persisted to a relational database and surfaced through an admin dashboard.

| # | Stage | Description |
|---|---|---|
| 1 | Life Event Detection | Analyses transaction categories and amounts to identify the dominant life event (home buying, marriage, education, medical, travel, vehicle purchase). |

| | | |
|---|---|---|
| 2 | Credit Eligibility | Scores the customer on transaction regularity, account balance, income level, and average spend. Computes maximum loan amount and applicable interest rate. |
| 3 | Product Selection | Maps the detected life event to the optimal product type, then filters by credit eligibility and selects the best-fit product (lowest rate for loans). |
| 4 | AI Message Generation | Calls Google Gemini 1.5 Flash with a structured prompt containing user profile, life event, and product data. Returns a plain-text, compliance-safe offer message. |
| 5 | Compliance Validation | Runs 6 automated checks: user ID, transaction data, product eligibility, credit limit, interest rate disclosure, and T&C disclosure. Classifies as APPROVED / PENDING / REJECTED. |
| 6 | Persistence & Analytics | Saves the offer (product, message, propensity score, channel, click/convert flags) to Supabase. Dashboard aggregates conversion rates and propensity scores. |

*Table 2: Six-Stage AI Analysis Pipeline*

# 4. System Architecture

## 4.1 Frontend — React + Vite

The admin dashboard is built with React (Vite bundler) and provides three primary views:

- **Dashboard:** Aggregate KPIs — total users, offers sent, conversion rate, average propensity score.
- **Users List:** All customers with offer counts and conversion rates fetched from Supabase.
- **User Detail:** Individual customer profile, transaction history, and AI-generated offer with compliance status.

## 4.2 Backend — FastAPI

The backend is a modular FastAPI application with a clean service-layer architecture. Each concern is isolated into its own service class, making the system testable and extensible.

| Module | File | Responsibility |
|---|---|---|
| API Router | analyze_router.py | Orchestrates the full pipeline; exposes REST endpoints for analysis, users, transactions, offers, and stats. |
| Life Event Service | life_event_service.py | Counts transaction categories, computes amount ratios, maps to LifeEventType enum, calculates confidence score. |
| Credit Eligibility | credit_eligibility_service.py | Scores base 600 plus factors for transaction count, balance, income, and average spend. Caps at 850. Computes max loan and interest rate. |
| Product Selection | product_selection_service.py | Maps life event to product type, filters by eligibility, selects lowest-rate loan or best-fit product. |
| Message Generator | message_generator.py | Lazily initialises Gemini 1.5 Flash, builds structured prompt, runs async in thread pool, cleans markdown from response. |

| | | |
|---|---|---|
| Compliance Service | compliance_service.py | Runs 6 checks, classifies APPROVED / PENDING / REJECTED based on critical vs non-critical failures. |
| Database | database.py | Supabase client initialisation; used by router for all database reads and writes. |

*Table 3: Backend Module Breakdown*

### 4.3 Database — Supabase (PostgreSQL)

Four core tables power the platform:

| Table | Key Columns | Purpose |
|---|---|---|
| users | id, name, age, income, credit_score, risk_category, marital_status, preferred_channel | Customer master data |
| transactions | id, user_id, merchant, amount, category, timestamp | Raw transaction history |
| products | id, name, product_type, interest_rate, max_amount, eligibility_criteria | Financial product catalogue |
| offers | id, user_id, product_id, life_events, propensity_score, generated_message, channel, clicked, converted | Offer log and engagement tracking |

*Table 4: Supabase Database Schema*

## 5. Component Deep-Dives

### 5.1 Life Event Detection

The **LifeEventService** analyses a list of *TransactionData* objects. It first aggregates total spend per category, then identifies the dominant category by highest cumulative amount. The category is mapped to a *LifeEventType* enum via a lookup table.

Confidence is computed as a weighted average of the amount ratio (spend in dominant category / total spend) and the count ratio (transactions in dominant category / total transactions). If a valid life event is detected, confidence is boosted by +0.30 and capped at 1.0.

| Life Event | Triggered By | Recommended Product |
|---|---|---|
| Home Buying | home_purchase category | Home Loan |
| Marriage | wedding category | Wedding Loan / Joint FD |
| Education Planning | education category | Education Loan |
| Medical Emergency | medical category | Health Insurance / Personal Loan |
| Vacation Planning | vacation category | Travel Credit Card |
| Vehicle Purchase | automotive category | Auto Loan |

*Table 5: Life Event to Product Mapping*

## 5.2 Credit Eligibility Engine

The **CreditEligibilityService** computes a synthetic credit score starting from a base of 600, adding points across four dimensions:

| Factor | Condition | Points |
|---|---|---|
| | 10 or more transactions | +50 |
| Transaction Regularity | 5 or more transactions | +30 |
| | 3 or more transactions | +10 |
| | Rs. 1,00,000 or above | +100 |
| Account Balance | Rs. 50,000 or above | +60 |
| | Rs. 10,000 or above | +30 |
| | Rs. 1,00,000 or above | +80 |
| Annual Income | Rs. 75,000 or above | +50 |
| | Rs. 50,000 or above | +30 |
| | Rs. 10,000 or above | +40 |
| Average Transaction Amount | Rs. 5,000 or above | +20 |
| | Rs. 1,000 or above | +10 |

*Table 6: Credit Score Computation Factors (Base: 600, Cap: 850)*

Interest rates are tiered by score: **750 and above: 6.0%**, **700 and above: 8.0%**, **650 and above: 10.0%**, **below 650: 12.0%**. Maximum loan amount is 3 to 5 times annual income depending on score tier.

## 5.3 Gemini AI Message Generation

The **MessageGenerationService** uses Google Gemini 1.5 Flash for low-latency inference. The model is lazily initialised and reused across requests. The prompt is carefully engineered to:

- Output plain text only — no markdown, bullets, asterisks, or emojis.
- Mention the product name and its primary benefit naturally.
- Connect the offer to the customer's detected life event or spending behaviour.
- Use a warm, professional tone appropriate for direct customer communication.
- Stay compliant: no guaranteed approvals, no unrealistic promises.

The Gemini call is executed asynchronously via *asyncio.to_thread()* to avoid blocking the FastAPI event loop. A post-processing step strips any residual markdown artifacts from the response.

## 5.4 Compliance Validation

The **ComplianceService** runs six checks on every offer before it is persisted or returned. Critical failures (user ID, transaction data, product eligibility) result in REJECTED status; non-critical failures result in PENDING; all checks passing results in APPROVED.

| Check | Type | Pass Condition |
|---|---|---|
| User ID Validation | Critical | user_id is non-empty |
| Transaction Data Validation | Critical | At least one transaction present |

| | | |
|---|---|---|
| Product Eligibility Check | Critical | credit_eligibility.is_eligible is True |
| Credit Limit Validation | Non-Critical | max_loan_amount is within product limit |
| Interest Rate Disclosure | Non-Critical | product.interest_rate is not None |
| Terms and Conditions Disclosure | Non-Critical | product.eligibility_criteria is non-empty |

*Table 7: Compliance Checks (Red = Critical, Amber = Non-Critical)*

# 6. REST API Reference

| Method | Endpoint | Description |
|---|---|---|
| POST | /analyze | Run full AI pipeline for a new request payload |
| POST | /analyze/users/{user_id} | Trigger AI analysis for an existing Supabase user |
| GET | /analyze/customer | Fetch primary customer profile (Supabase or mock fallback) |
| GET | /analyze/transactions | Fetch recent transactions (Supabase or mock fallback) |
| GET | /analyze/users | List all users with offer counts and conversion rates |
| GET | /analyze/users/{user_id} | Get single user detail |
| GET | /analyze/users/{user_id}/transactions | Get all transactions for a specific user |
| GET | /analyze/users/{user_id}/offers | Get all offers for a user (includes product name) |
| GET | /analyze/stats | Dashboard aggregates: users, offers, conversion rate, average propensity |
| GET | /health | Health check endpoint — returns status: healthy |
| GET | /docs | Auto-generated Swagger UI (FastAPI built-in) |

*Table 8: REST API Endpoints*

# 7. Technology Stack

| Layer | Technology | Notes |
|---|---|---|
| AI / LLM | Google Gemini 1.5 Flash | Via google-generativeai SDK; async thread execution to avoid blocking the event loop |
| Backend Framework | FastAPI | Python 3.11+; async/await throughout; Pydantic v2 request/response schemas |
| Database | Supabase (PostgreSQL) | Managed cloud database; real-time subscriptions available for future use |

| | | |
|---|---|---|
| Frontend | React + Vite | Modular page structure (Dashboard, Users, User Detail); API calls via fetch |
| Database Client | supabase-py | Python client for Supabase REST API |
| Data Validation | Pydantic v2 | Strict schema enforcement on all request and response models |
| Environment Config | python-dotenv | GEMINI_API_KEY, SUPABASE_URL, SUPABASE_KEY loaded from .env file |
| CORS | FastAPI CORSMiddleware | Configured for all origins in development; restrict to specific origins in production |
| Deployment | Render / Railway | runtime.txt specifies Python version for cloud deployment |

*Table 9: Full Technology Stack*

# 8. Setup and Deployment

## 8.1 Backend Setup

```
git clone <repo-url> and cd technova/backend
python -m venv venv and venv\Scripts\activate (Windows)
pip install -r requirements.txt
Create .env file with: GEMINI_API_KEY=... SUPABASE_URL=... SUPABASE_KEY=...
uvicorn main:app --reload --port 8000
Visit http://localhost:8000/docs for interactive API documentation
```

## 8.2 Frontend Setup

```
cd technova/frontend
npm install
npm run dev
Visit http://localhost:5173 for the admin dashboard
```

## 8.3 Environment Variables

| Variable | Description | Required |
|---|---|---|
| GEMINI_API_KEY | Google AI Studio API key for Gemini 1.5 Flash | Yes |
| SUPABASE_URL | Supabase project URL (https://xxx.supabase.co) | Yes |
| SUPABASE_KEY | Supabase anonymous or service role key | Yes |
| GEMINI_MODEL_NAME | Override model name (default: gemini-1.5-flash) | No |

*Table 10: Required Environment Variables*

# 9. Innovation and Key Differentiators

| Feature | FinSight Approach | Typical Bank System |
|---|---|---|
| Offer Timing | **Real-time, triggered by transaction** | Batch campaigns (weekly/monthly) |
| Personalisation | **Life event + credit profile + AI message** | Segment-based generic templates |
| Risk Assessment | **Multi-factor scoring (4 dimensions)** | Single credit bureau score |
| Message Generation | **Gemini AI — unique per customer** | Fixed template with name substitution |
| Compliance | **6 automated checks per offer** | Manual review or none |
| Feedback Loop | **Click/convert tracked per offer in DB** | Aggregate campaign metrics only |
| Extensibility | **Modular service classes — easy to extend** | Monolithic rule engine |

*Table 11: FinSight vs. Typical Bank System (Green = FinSight advantage)*

## 10. Future Roadmap

| Phase | Feature | Impact |
|---|---|---|
| **v1.1** | WhatsApp / SMS delivery integration | Reach customers on their preferred channel |
| **v1.2** | A/B testing for offer messages | Data-driven message optimisation |
| **v1.3** | Real-time credit bureau API integration | Replace synthetic score with live CIBIL data |
| **v2.0** | Reinforcement learning feedback loop | Model improves with each click and convert signal |
| **v2.1** | Multi-language offer generation | Serve regional language customers (Hindi, Tamil, etc.) |
| **v2.2** | Fraud signal integration | Block offers to flagged accounts automatically |
| **v3.0** | Open Banking API support | Aggregate data across multiple bank accounts |

*Table 12: Product Roadmap*

## 11. Team — Meow Meow

Team Meow Meow is participating in InnovGenius 2026, the TCS Hackathon, under Problem Statement PS-02. The team built the FinSight Cross-Sell Platform end-to-end — from database schema design and FastAPI backend to the React admin dashboard and Gemini AI integration.

| Role | Contribution |
|---|---|
| Backend / AI | FastAPI service architecture, Gemini integration, Supabase schema design, compliance engine |
| Frontend | React dashboard, user / transaction / offer views, API integration |
| Data / ML | Life event detection logic, credit scoring model, propensity score calculation |

| | |
|---|---|
| DevOps / QA | Deployment configuration, environment setup, API testing and validation |

*Table 13: Team Meow Meow — Roles and Contributions*